ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

Module 9: Continuous Integration (CI)

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Introduction to Continuous Integration (CI)

- What **Continuous Integration (CI)** means in software development
- How CI fits into the DevOps lifecycle
- Why CI is essential for backend and DevOps engineers
- Relationship between **CI** and **CD** (preview)

What Is Continuous Integration?

- **Continuous Integration**:
  - Practice of automatically building and testing code whenever changes are integrated into a shared repository
- Core idea:
  - Integrate **early** and **often**
- Typical cycle:
  - Developer pushes code → CI pipeline runs → feedback in minutes

Problems CI Solves

- Without CI:
  - **Inconsistent builds**:
    - “It builds on my machine but not on yours”
  - **Slow manual testing**:
    - Tests run only before big releases
  - **Unstable branches**:
    - main/master often broken
  - **Late feedback**:
    - Bugs detected weeks after they were introduced
- CI aims to:
  - Catch issues **as soon as they are introduced**

High-Level DevOps Pipeline

- Typical pipeline stages:
  - **Commit** → **Build** → **Test** → **Analyze** → **Artifact** → (CD)
- CI focuses on:
  - Build
  - Test
  - Static analysis and quality checks
- CD (next modules) focuses on:
  - Deploying artifacts to staging and production

Preview: Continuous Delivery / Deployment (CD)

- **Continuous Delivery (CD)**:
  - Ability to deploy a build to production at any time
  - Deployment is a **manual decision**, but always ready
- **Continuous Deployment**:
  - Every change that passes CI is **automatically** deployed to production
- Both require:
  - **Strong CI pipelines** as a foundation

Scope of This Module

- This module covers:
  - Principles and architecture of **CI**
  - GitHub Actions as our main CI tool
  - Building CI pipelines for Node.js and Spring Boot
  - Quality gates and Docker image builds
- CD will be:
  - Covered in detail in the next module
  - CI configuration will be reused and extended

Core Principles of Effective CI

- Trunk-based development and stable mainline
- Small, frequent commits
- Automation of build, test, and analysis
- Fast feedback and fail-fast mindset
- Reproducible builds and Docker

Single Stable Mainline

- Maintain a single **stable main** branch:
  - All production releases come from main
  - main is always **green** (CI passing)
- Feature branches:
  - Short-lived
  - Frequently merged via Pull Requests
- Benefits:
  - Reduced merge conflicts
  - More predictable releases

Small, Frequent Commits

- Commit strategy:
  - Make small, logical changes
  - Push them frequently
- Advantages:
  - Easier to review
  - Easier to rollback
  - CI pipeline runs faster on smaller diffs
- Anti-pattern:
  - Giant “end of week” commit that breaks everything

Automate Everything

- CI should automate:
  - **Build**:
    - `npm ci`, `./mvnw package`
  - **Tests**:
    - Unit and integration tests
  - **Linting and formatting**:
    - ESLint, Prettier, Checkstyle
  - **Security checks**:
    - Dependency scanning, SAST
- Goal:
  - One push → entire pipeline runs with no manual steps

Fast Feedback Loops

- Good CI pipeline:
  - Finishes in **under 5–10 minutes** for typical changes
- Benefits:
  - Developers fix issues while context is fresh
  - Teams avoid “merge, then go home and hope”
- Techniques:
  - Parallel jobs
  - Test selection
  - Caching dependencies and Docker layers

Fail-Fast Approach

- If something is wrong:
  - Fail the pipeline **as early as possible**
- Examples:
  - Lint before build
  - Run fast unit tests before heavier integration tests
- Purpose:
  - Failures are visible quickly in Pull Requests
  - Developers do not waste time waiting for slow steps

Reproducible Builds with Docker

- Problem:
  - Different environments (OS, tools, versions)
- Docker-based builds:
  - CI pipeline runs inside standardized containers
  - Dockerfile defines exact runtime (Node.js/Java versions, tools)
- “Build once, test everywhere”:
  - Same Docker image built in CI can be:
    - Tested
    - Scanned
    - Deployed in multiple environments

Architecture of a CI Pipeline

- Typical CI pipeline stages:
  - **Trigger** → **Build** → **Test** → **Analyze** → **Artifact**
- Tools:
  - GitHub Actions, GitLab CI, Jenkins, etc.
- We treat the pipeline as **code** (YAML files)

CI Triggers

- Common trigger events:
  - `push`:
    - Every commit to specified branches
  - `pull_request`:
    - Every PR opened or updated
  - `schedule`:
    - Nightly builds and scans (cron-like)
  - `workflow_dispatch`:
    - Manual trigger from UI

CI Triggers

- Example (GitHub Actions):

```yaml
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

- The pipeline runs:
  - On every push to `main`
  - On every PR targeting `main`

Build Stage

- Node.js example:

```
npm ci
npm run build
```

- Spring Boot example:

```
./mvnw -B clean package
```

- Responsibilities:
  - Install dependencies
  - Compile application
  - Produce artifacts: Built JS bundle, JAR/WAR file, Docker image

Test Stage

- **Unit tests**:
  - Test individual functions/classes
- **Integration tests**:
  - Test interaction between components
- Example (Node.js):

```
npm test -- --coverage
```

- Example (Spring Boot):

```
./mvnw -B test
```

- Test results: Often exported as JUnit XML, coverage reports

Mocking External Services

- Real systems depend on:
  - Payment gateways
  - Email services
  - External APIs
- In CI:
  - Do **not** call real external services
- Instead:
  - Use mocks or test doubles
  - Spin up local dependencies in Docker (e.g., test database)

Code Quality Gates

- CI should enforce **quality gates**:
  - Linting, Formatting
  - Static analysis, Security scanning
  - Coverage thresholds
- Example:

```
npm run lint
npm run test -- --coverage --watch=false
```

- If any gate fails:
  - CI fails
  - Pull Request cannot be merged (with branch protection)

Artifact Storage (No Deployment Yet)

- CI produces **artifacts**:
  - Node.js build output folder (e.g., `dist/`)
  - Spring Boot JAR (e.g., `target/app.jar`)
  - Test coverage reports
  - Docker images pushed to registry
- Artifacts are:
  - Used later by CD pipeline
  - Available for manual download and testing

Overview of CI Tools & Ecosystem

- Popular CI tools:
  - GitHub Actions
  - GitLab CI/CD
  - Jenkins
  - CircleCI
  - AWS CodeBuild
- Choice depends on:
  - Repository hosting
  - Team preferences
  - Cloud provider

GitHub Actions (Primary Focus)

- Built directly into GitHub:
  - No separate server needed
- Key features:
  - YAML-based workflows
  - GitHub-hosted runners
  - Reusable actions from Marketplace
- Great for:
  - Student projects
  - Small to medium-sized teams
  - Open-source repositories

Other CI Tools (High-Level Overview)

- **GitLab CI/CD**:
  - Integrated with GitLab repositories
  - Similar YAML syntax (`.gitlab-ci.yml`)
- **Jenkins**:
  - Long-standing CI server
  - Highly customizable
  - Requires self-hosting and maintenance
- **CircleCI / Travis CI / CodeBuild**:
  - SaaS CI options
  - Good for specific ecosystems and integrations

SaaS vs Self-Hosted Runners

- **SaaS runners** (GitHub-hosted):
  - Managed infrastructure
  - Easy to start
  - Limited by per-minute quotas
- **Self-hosted runners**:
  - Run on your own machines/VMs
  - Can access private networks securely
  - Require maintenance and security hardening
- For this course:
  - We focus on **GitHub-hosted runners**

CI Using GitHub Actions

- Core concepts:
  - Workflows, jobs, steps, actions, runners, events
- YAML syntax and attributes
- Node.js and Spring Boot CI examples

GitHub Actions Core Concepts

- **Workflow**:
  - Top-level pipeline defined in YAML
- **Job**:
  - Group of steps that run on the same runner
- **Step**:
  - Individual task: checkout, run command, use an action
- **Action**:
  - Reusable unit of work from Marketplace or custom
- **Runner**:
  - Machine that executes jobs (e.g., `ubuntu-latest`)
- **Event**: Trigger that starts the workflow (push, PR, etc.)

GitHub Actions Workflow Structure

- Minimal Node.js workflow:

```yaml
name: Node.js CI

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
      - run: npm ci
      - run: npm test -- --coverage
```

YAML Syntax Refresher

- **Indentation matters**:
  - Use two spaces consistently
- **Lists**:

```yaml
steps:
  - name: First step
    run: echo "Hello"
  - name: Second step
    run: echo "World"
```

- **Key–value pairs**:

```yaml
runs-on: ubuntu-latest
name: Node.js CI
```

Common Workflow Attributes

- `name`:
  - Human-readable name in GitHub UI
- `on`:
  - Events that trigger the workflow
- `jobs`:
  - Map of job definitions
- Inside each job:
  - `runs-on`: type of runner VM
  - `steps`: list of actions and shell commands

Steps: uses vs run

- `uses`:
  - Invoke a reusable action:

```
- uses: actions/checkout@v4
- uses: actions/setup-node@v4
```

- `run`:
  - Run a shell command on the runner:

```
- run: npm ci
- run: npm test -- --coverage
```

- Combine both to build powerful pipelines

Environment Variables in GitHub Actions

- Define env at workflow or job level:

```yaml
env:
  NODE_ENV: test
  LOG_LEVEL: info
```

- Access in shell commands:

```bash
echo "$NODE_ENV"
```

- Example inside a job:

```yaml
jobs:
  build:
    runs-on: ubuntu-latest
    env:
      API_BASE_URL: https://api.test.mystore.com
```

Secrets and Expressions

- **Secrets**:
  - Store sensitive data securely in GitHub:
    - Tokens, Passwords, API keys
  - Access in YAML:

```yaml
env:
  DOCKER_PASSWORD: ${{ secrets.DOCKER_PASSWORD }}
```

- **Expressions**:
  - Use `${{ }}` syntax for context values:
    - `${{ github.sha }}`
    - `${{ github.ref_name }}`
    - `${{ matrix.node-version }}`

Strategy Matrix for Multiple Versions

- Test against multiple versions of Node.js:

```yaml
strategy:
  matrix:
    node-version: [16, 18, 20]
```

- Use in steps:

```yaml
- uses: actions/setup-node@v4
  with:
    node-version: ${{ matrix.node-version }}
```

- Benefit:
  - Ensure compatibility across runtimes

Node.js CI Pipeline Using GitHub Actions

- Main tasks:
  - Checkout code
  - Setup Node.js
  - Install dependencies with `npm ci`
  - Run tests with coverage
  - Run linter
  - Cache dependencies for speed

Node.js CI Workflow Example

```yaml
name: Node.js CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: "npm"
      - name: Install dependencies
        run: npm ci
      - name: Run tests with coverage
        run: npm test -- --coverage
      - name: Run ESLint
        run: npm run lint
```

Explaining the Node.js Workflow

- `actions/checkout@v4`:
  - Checks out repository code into the runner
- `actions/setup-node@v4`:
  - Installs Node.js 18
  - Enables automatic NPM cache restoration
- `npm ci`:
  - Clean, reproducible install using `package-lock.json`
- `npm test -- --coverage`:
  - Runs tests, collects coverage
- `npm run lint`:
  - Enforces code quality rules

Adding Caching Manually (When Needed)

- Explicit caching example:

```yaml
- name: Cache npm
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      npm-
```

- Speeds up:
  - Subsequent installs
- Important:
  - Key changes when `package-lock.json` changes

Uploading Coverage Reports as Artifacts

```yaml
- name: Run tests with coverage
  run: npm test -- --coverage

- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: coverage/
```

- Artifacts:
  - Downloadable from GitHub Actions UI
  - Useful for manual inspection or integration with other tools

Spring Boot CI Pipeline Using GitHub Actions

- Similar structure to Node.js:
  - Checkout
  - Setup JDK
  - Build and test with Maven
  - Upload JAR artifact
  - Optional caching of Maven dependencies

Spring Boot CI Workflow Example

```yaml
name: Spring Boot CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup JDK
        uses: actions/setup-java@v4
        with:
          distribution: "temurin"
          java-version: "17"
          cache: "maven"
      - name: Run tests
        run: ./mvnw -B clean test
      - name: Package application
        run: ./mvnw -B package
      - name: Upload JAR artifact
        uses: actions/upload-artifact@v4
        with:
          name: spring-boot-app
          path: target/*.jar
```

Explaining the Spring Boot Workflow

- `distribution: "temurin"`:
  - Use Eclipse Temurin JDK distribution
- `java-version: "17"`:
  - Standard LTS version for modern Spring apps
- `cache: "maven"`:
  - Automatically caches Maven dependencies
- `./mvnw -B clean test`:
  - Batch mode (no interactive prompts)
  - Clean previous outputs, run tests
- `./mvnw -B package`:
  - Builds the production JAR file

Configuring Maven Caching Explicitly

- When not using built-in caching:

```yaml
- name: Cache Maven repository
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: |
      maven-
```

- Benefit:
  - Faster builds, especially with many dependencies

Static Analysis, Security Scanning & Quality Gates

- CI is also about **code quality** and **security**
- We integrate:
  - Linting and formatting
  - Type checking
  - Security scanning
  - Coverage thresholds

Linting and Formatting

- Node.js:
  - ESLint and Prettier:

```
npm run lint
npm run format:check
```

- Java:
  - Checkstyle, Spotless:

```
./mvnw checkstyle:check
./mvnw spotless:check
```

- Pipelines should:
  - Fail when style rules are violated

Test Coverage Thresholds

- Ensure tests are not only present but **cover enough code**:
  - Node.js (Jest):

```
"jest": {
  "collectCoverage": true,
  "coverageThreshold": {
    "global": {
      "branches": 80,
      "functions": 80,
      "lines": 80
    }
  }
}
```

- CI failure:
  - If coverage is below the threshold, tests fail → pipeline fails

Dependency Scanning and GitHub Dependabot

- **Dependabot**:
  - Automatically creates PRs to update vulnerable dependencies
- CI integration:
  - Run tests and lint on Dependabot PRs
  - Ensure upgrades do not break builds
- Additional scanners:
  - `npm audit`
  - `mvn versions:display-dependency-updates`

Security Scanning and SAST (Preview)

- **SAST (Static Application Security Testing)**:
  - Analyzes source code for security issues
- GitHub CodeQL (example job):

```yaml
- name: Initialize CodeQL
  uses: github/codeql-action/init@v3
  with:
    languages: javascript
- name: Perform CodeQL Analysis
  uses: github/codeql-action/analyze@v3
```

- SonarCloud: Cloud-based code quality and security analysis
- CI should: Fail builds if critical vulnerabilities are found

CI with Docker (No Deployment Yet)

- Building Docker images in CI:
  - Ensures builds are reproducible
  - Prepares images for CD
- Steps:
  - Build image
  - Tag with `latest` and `github.sha`
  - Push to container registry (e.g., Docker Hub)

Building Docker Images in GitHub Actions

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}
- name: Build Docker image
  run: docker build -t mvmanh/node-api:${{ github.sha }} .
- name: Tag image as latest
  run: docker tag mvmanh/node-api:${{ github.sha }} mvmanh/node-api:latest
- name: Push images
  run: |
    docker push mvmanh/node-api:${{ github.sha }}
    docker push mvmanh/node-api:latest
```

- Secrets: DOCKER_USERNAME, DOCKER_PASSWORD stored in repository settings

Multi-Stage Docker Builds in CI

- Dockerfile example:

```
FROM node:20-alpine AS build
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /usr/src/app
COPY --from=build /usr/src/app/dist ./dist
CMD ["node", "dist/server.js"]
```

- Benefits: Smaller runtime image; Faster deployments

Caching Docker Layers

- Use Docker BuildKit and cache:

```yaml
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push with cache
  uses: docker/build-push-action@v6
  with:
    push: true
    tags: mvmanh/node-api:${{ github.sha }}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

- Result:
  - Future builds reuse layers and run faster

Collaboration Workflow & Branch Protection

- CI is not only about machines; it shapes **team workflow**
- Combine:
  - Pull Request review
  - Branch protection rules
  - CI checks

PR-Based Development Workflow

- Typical flow:
  i. Create feature branch
  ii. Commit and push changes
  iii. Open Pull Request to `main`
  iv. CI runs automatically on PR
  v. Reviewers check code and CI status
  vi. Merge only if CI passes and review is approved
- This preserves:
  - Quality and stability of main branch

Branch Protection Rules (GitHub)

- In repository settings:
  - Protect `main` branch:
    - Require status checks to pass
    - Require pull request reviews
    - Block force pushes
- Example status checks:
  - `Node.js CI (build)`
  - `Spring Boot CI (build)`
- Effect:
  - You cannot merge a PR if CI is red

Using Status Checks to Prevent Broken Code

- In a PR:
  - Each job appears as a status check
- If tests fail:
  - Status check is red
  - Merge button is disabled
- Encourages:
  - Fixing tests and lint errors before merging
  - Treating CI failures as **team priority**

Practical Hands-On Labs (CI-Only)

- Lab 1: Node.js CI workflow
- Lab 2: Spring Boot CI workflow
- Lab 3: CI + Docker build
- Lab 4: Enforcing PR quality gates

Lab 1 – Node.js CI Workflow

- Goals:
  - Create a `ci-node.yml` workflow
  - On every push/PR:
    - Install dependencies
    - Run tests with coverage -> Run ESLint
    - Upload coverage report as artifact
- Key steps:
  - Use `actions/checkout@v4`
  - Use `actions/setup-node@v4`
  - Use `actions/upload-artifact@v4`

Lab 2 – Spring Boot CI Workflow

- Goals:
  - Create `ci-spring.yml`
  - On every push/PR:
    - Setup JDK 17
    - Run Maven tests
    - Package JAR
    - Upload JAR artifact
- Extensions:
  - Add Maven dependency caching
  - Enforce `mvn test` success before packaging

Lab 3 – CI + Docker Build Pipeline

- Goals:
  - Create `ci-docker.yml`
  - On every push to `main`:
    - Build Docker image for Node.js API
    - Tag image with `github.sha` and `latest`
    - Push to Docker Hub
- Skills:
  - Using Docker login action
  - Handling sensitive credentials via GitHub secrets
  - Managing tags for traceability

Lab 4 – Enforcing PR Quality Gates

- Steps:
  - Configure branch protection on `main`
  - Require:
    - CI checks to pass
    - At least one code review approval
  - Create PRs that:
    - Break tests, Fail lint
    - Fix issues until CI passes
- Outcome:
  - Students see how CI prevents low-quality code from being merged

Best Practices for CI Pipelines

- Keep pipelines fast and focused
- Use caches aggressively
- Fail early, fail loud
- Keep workflows clean and modular
- Avoid duplication with reusable workflows and composite actions

Keeping CI Fast

- Target:
  - < 5 minutes for typical PRs
- Strategies:
  - Separate fast and slow tests
  - Run heavy integration tests on schedule or in dedicated jobs
  - Use caching for dependencies and Docker layers
- Regularly monitor:
  - CI performance metrics and optimize

Modular and Reusable Workflows

- Problem:
  - Copy–paste the same YAML across multiple repos
- Solutions:
  - **Reusable workflows**:
    - `workflow_call` mechanism in GitHub Actions
  - **Composite actions**:
    - Wrap repeated logic into a custom action
- Benefits:
  - Less duplication
  - Easier maintenance

Logging and Debugging Information

- Collect detailed logs:
  - Use `--info` or `--debug` flags (careful with secrets)
- Store:
  - Test reports
  - Coverage reports
  - Build logs (if necessary)
- Make debugging easier:
  - Include links to relevant logs in PR comments

Troubleshooting CI Failures

- Common issues:
  - Caching problems
  - “Works locally but fails in CI”
  - Environment differences
  - Flaky tests
  - Timeouts and rate limits

Dependency Caching Issues

- Symptoms:
  - Cache not being restored
  - Very slow `npm ci` or `mvn` every run
- Check:
  - Cache key definitions (`hashFiles` paths)
  - Restore keys
  - Cache size limits
- Fix:
  - Narrow or adjust cache paths
  - Make cache keys more specific to lockfiles

Works Locally but Fails in CI

- Possible causes:
  - CI uses Linux, developer uses Windows/macOS
  - Missing environment variables in CI
  - Hidden dependencies on local tools (e.g. global NPM packages)
- Troubleshooting:
  - Reproduce CI environment locally using Docker:

    ```
    docker run -it --rm ubuntu:22.04 bash
    ```

  - Install same tools as CI
  - Run the same commands from the workflow

Handling Flaky Tests

- Flaky test:
  - Sometimes passes, sometimes fails with no code changes
- Strategies:
  - Identify and fix root cause:
    - Race conditions
    - Timeouts
    - External dependencies
  - Temporarily quarantine flaky tests:
    - Mark as `@flaky` and run in separate job
- CI principle:
  - Test results should be **deterministic**

Runner Timeouts and Resource Limits

- Timeouts:
  - Workflows may hit default time limit (e.g., 60 minutes)
- Resource limits:
  - Memory or disk exhaustion
- Solutions:
  - Split jobs into smaller pieces
  - Use more powerful self-hosted runners for heavy workloads
  - Optimize tests and build steps

Rate Limits (npm, Maven, GitHub API)

- During peak times:
  - `npm` or Maven repositories may throttle requests
  - GitHub API calls in actions may be rate-limited
- Mitigation:
  - Use official registries with authentication (to increase quotas)
  - Configure mirrors or local proxies
  - Reduce unnecessary API calls

Summary and Takeaways

- CI is the **foundation** of modern DevOps
- Good CI pipeline:
  - Is automated, fast, and reliable
  - Enforces quality and security gates
  - Produces artifacts ready for CD
- Next module:
  - Build on this foundation to implement **Continuous Delivery / Deployment (CD)**